



Recap: Computational Complexity



1. Use **running time** as an indicator.
2. Use **number of operations or steps** as an indicator.
 - Represented by $T(n)$
3. Use asymptotic notation as an indicator.
 - **Big-O notation**

| $O(1)$ - Constant Time

Task: Pick up the "Top paper of the Stack"

Imagine grabbing the top paper off a stack. It takes the same amount of effort whether the stack is 1 inch high or 1 mile high.

```
item = prices[0]
```

Key Takeaway: Instant access. Zero growth in time.



| $O(\log n)$ - Logarithmic Time

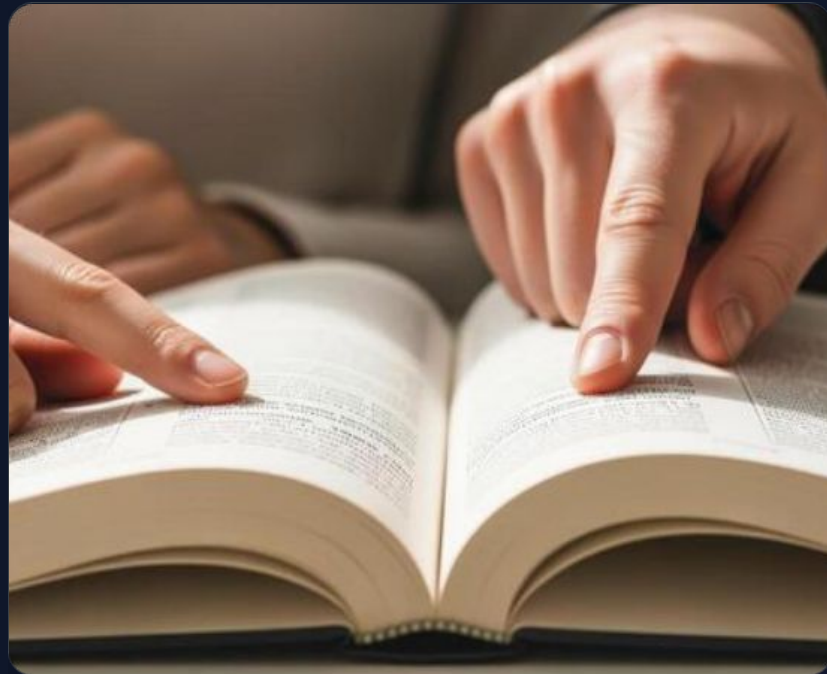
Task: Finding a page in a book

Imagine finding a page in a book.

You open the middle. If the page is too high, you ignore the second half. You keep splitting the problem in half.

```
# Binary Search  
while low <= high: ...
```

Key Takeaway: Extremely efficient. 1,000,000 items takes only ~20 steps.



O(n) - Linear Time

Task: The "Receipt Check"

Imagine reading a receipt to find a mistake.

You must check line 1, then line 2, then line 3,

all the way to the end. You cannot skip anything.

```
for item in prices:  
    print(item)
```

Key Takeaway: Fair growth. If data doubles, time doubles.



AD CNTRY SLC	3.49	F
D BEEF 30% 2LB	7.98	F
UL ONIONS 12PK	3.99	F
ATOS VALU 6CT	2.99	F
KDNY BEANS 34OZ	2.29	F
LCU ZSTY SPCE	1.75	F
2 FOR 3.50	1.75	F
H GUAC	4.99	F
LMAN'S MAYO 30OZ	3.99	F
AINE HRTS 4CT	3.49	F
RDEE TRASH BAGS	8.99	F
ATOS PRT SIZE	1.49	F
ES BK	1.98	F
2 FOR 0.99	1.98	F
K CAPN CRUNCH	4.99	F
COCUP PDN 12PK	5.49	F
ET MX CUP TROP 12CT	7.99	F
ROAST BEEF MED	11.99	F
D LB & 7.99/ 1b	11.60	F
DELI HAM NTFREE	11.60	F
D LB & 5.80/ 1b	11.60	F
CUIITS BLK PEP	2.25	F
2 FOR 4.50	2.25	F
L PAPER TOWELS 24CT	19.99	F
DRELLE DINR FMLY	10.99	F
CHURN VANILLA	5.49	F

O(n log n) - Linearithmic Time

Task: Card Sorting

Imagine sorting a deck of cards.

You have to touch every card (n), and you shuffle through them multiple times ($\log n$) to get them in order.

```
prices.sort()
```

Key Takeaway: Standard for sorting. Slower than $O(n)$, but necessary for ranking.



$O(n^2)$ - The Danger Zone

Task: The "Handshake Problem"

Imagine a networking event.

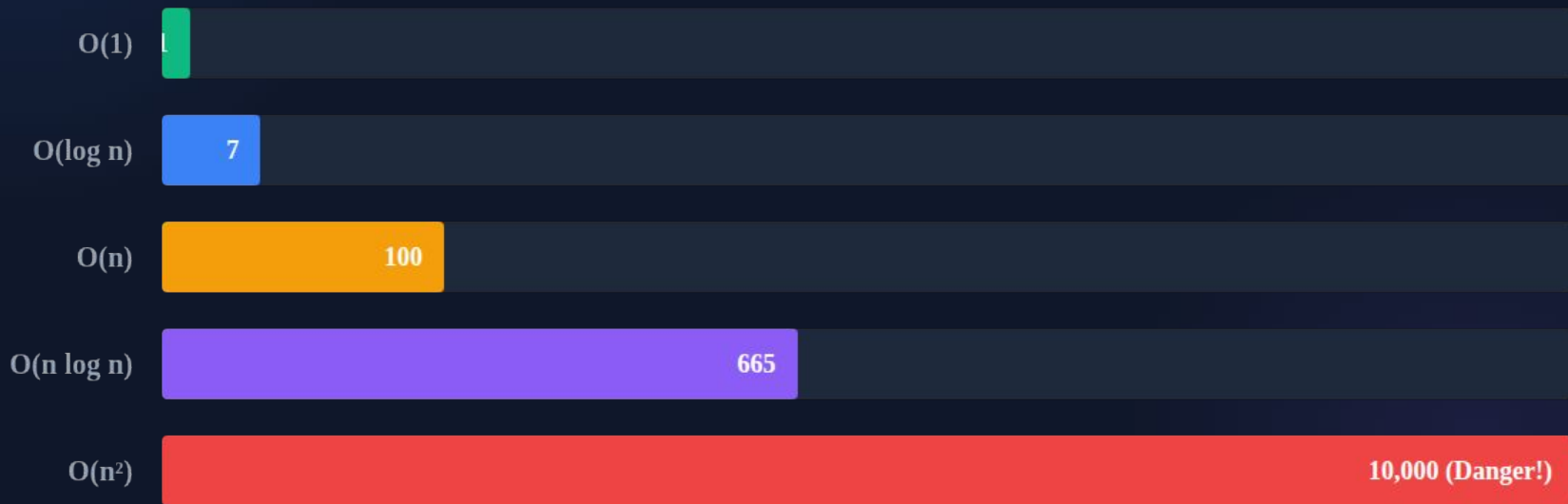
If everyone has to shake hands with everyone else, the number of handshakes explodes as more people arrive.

```
for x in list:  
    for y in list: ...
```

Warning: Doubling input size quadruples the run time. Avoid for large datasets.



Scalability Summary (N=100)



Comparing the number of operations required for an input size of just 100 items.



Chapter 2: Arrays



Dr. Sirasit Lochanachit



Today's Outline



1. What is an Array?
2. Static vs Dynamic Array
3. Array Operations and Asymptotic Performance



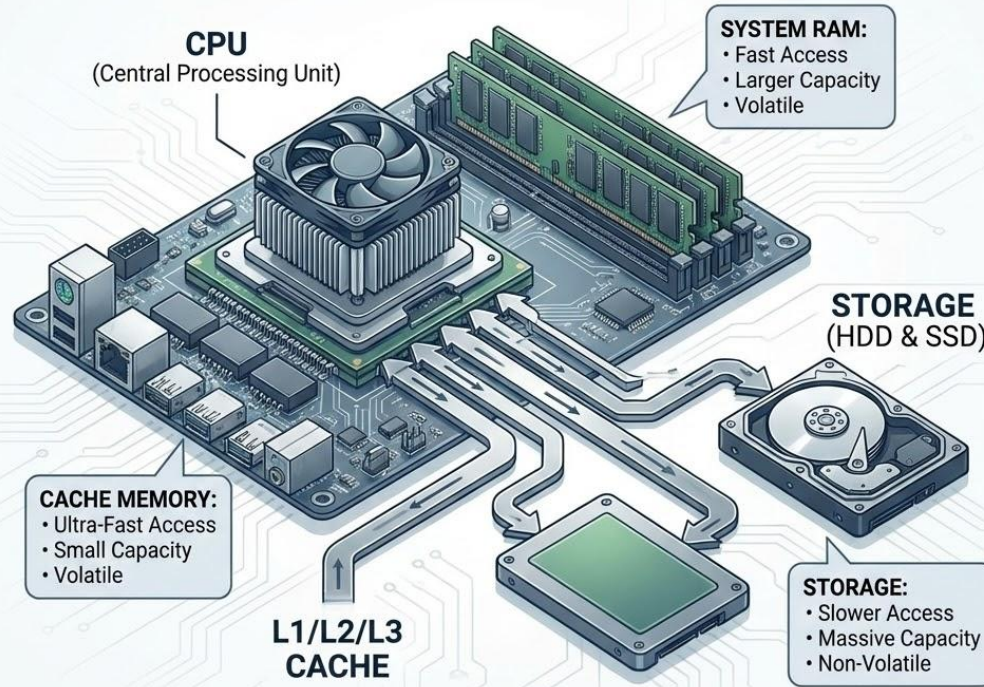
Python Sequence Types



- Built-in class
 - list
 - tuple
 - str

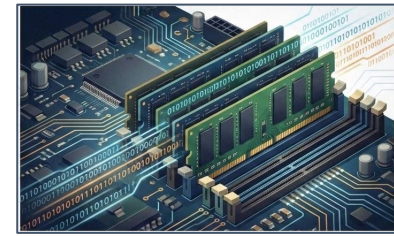
What are the characteristics shared amongst these 3 classes?

COMPUTER MEMORY: COMPONENTS & HIERARCHY





Computer Memory



- A computer will have a large number of bytes of memory.



Computer Memory



Memory Address Tracking:
Stores data with a unique location for easy retrieval.

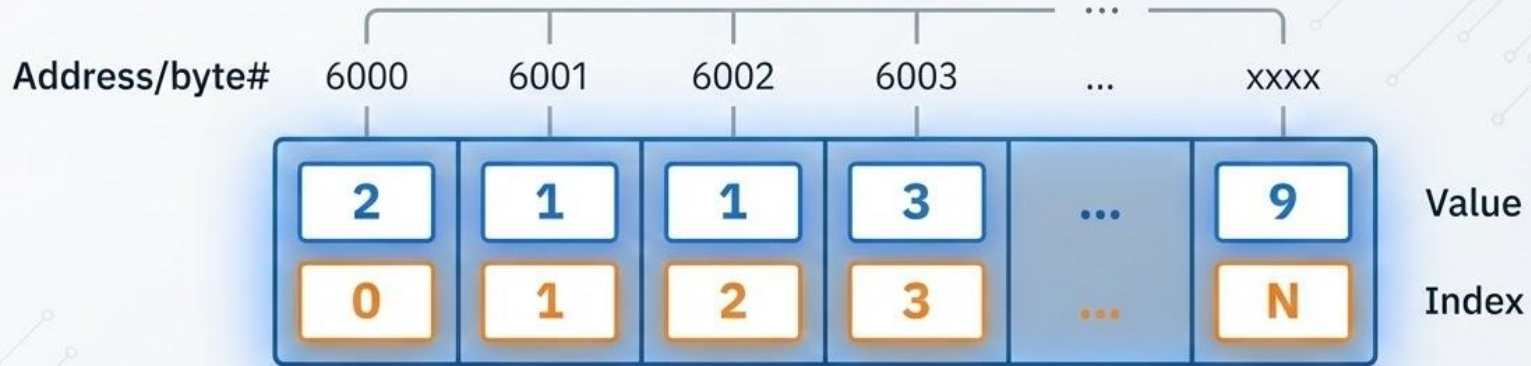


Unique Byte Addressing:
Each byte is assigned a distinct, sequential numerical address.



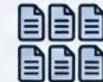
Constant Time Access:
Any byte can be read or written instantly, regardless of its position, with $O(1)$ time complexity.

What is an Array?



An array is a **chunk of memory**, consisting of **equal-size** elements.

$\begin{matrix} 1 \\ 2 \\ 3 \end{matrix}$ — Each of those elements has an **integer index**, which uniquely refers to the value stored.



The values are all of the **same type** (integer, character, etc.).

Array of Characters



In Python, it represents a **Unicode character** with 16 bits (i.e., 2 bytes).



Since each cell has an equal-size, any element can be accessed constantly with a formula:
 $\text{start_address} + (\text{cell_size} * \text{index})$



Exercise: How computer can access any element

Given an array:



Start Address: 6000

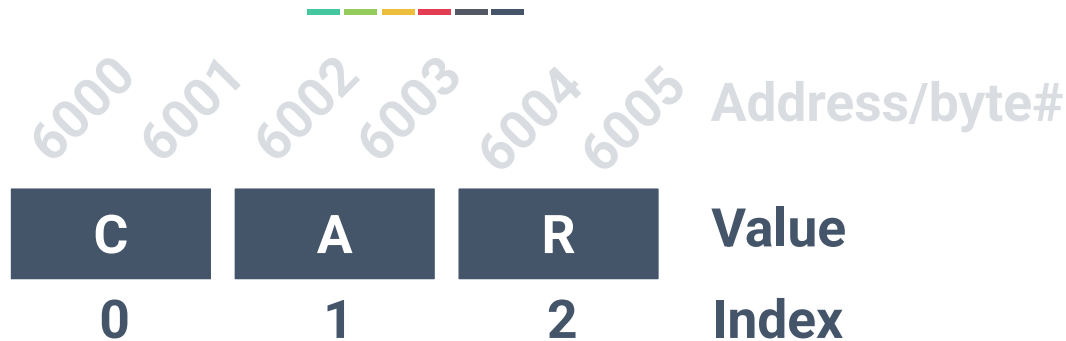


Cell Size: 8

What is the address of the element at index 6?



Array of Characters



- Luckily, a programming language calculate memory addresses of an array automatically.

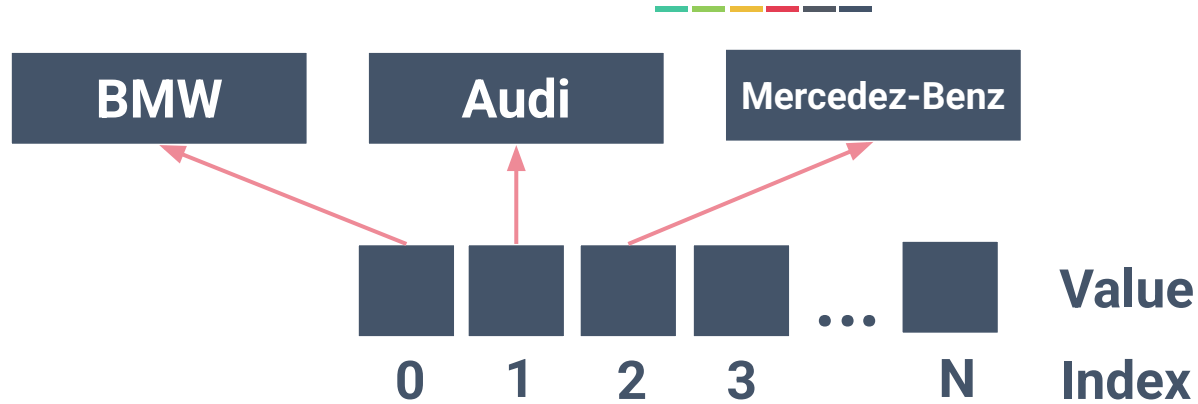


Array of Strings?



- Can we use an array to store strings of characters?

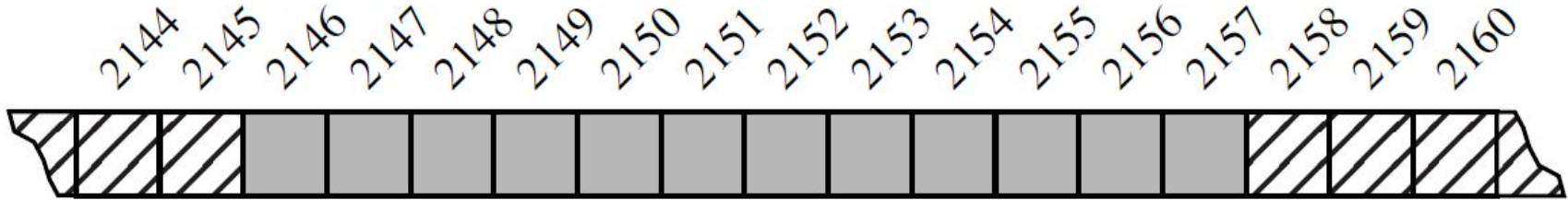
Python List



- Python list is a referential-type array that stores the memory addresses (references) of a value instead of the value itself.
- Strings can be in any length, but memory addresses are fixed-size.



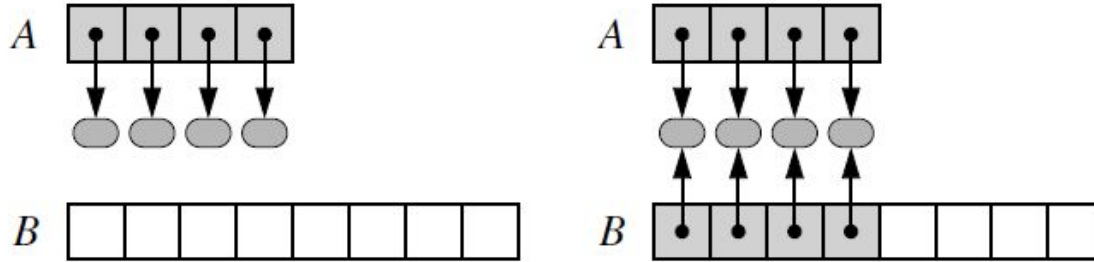
Static Array



- Once an array in C or Java has been created, it's size is fixed.
- Python tuple and str instances are immutable.
 - Unable to change size and value

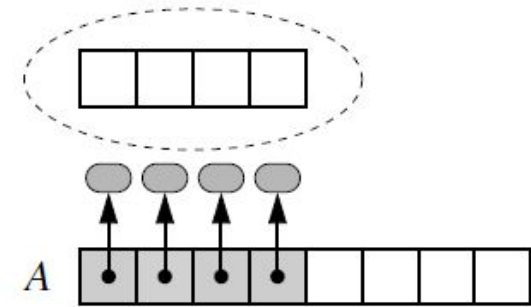
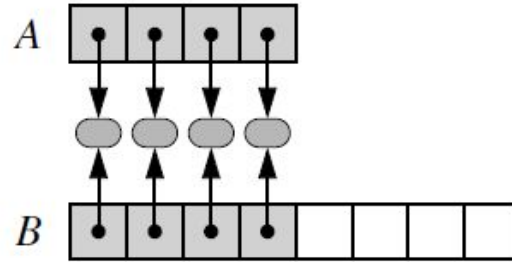
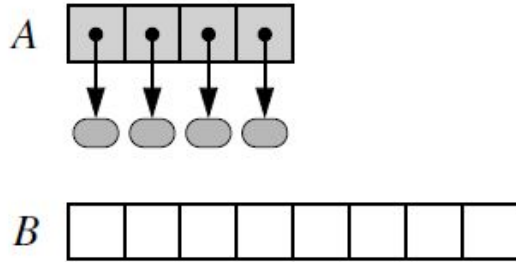


Dynamic Array



- Python's list instance maintains an underlying array that often has greater capacity than the current length of the list.
- If a capacity is exhausted, the list class requests a new, larger array.

Dynamic Array



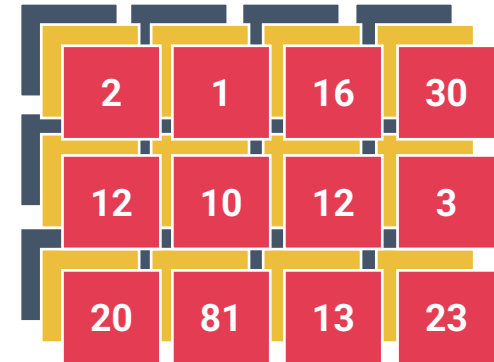
Arrays



(a) One dimension

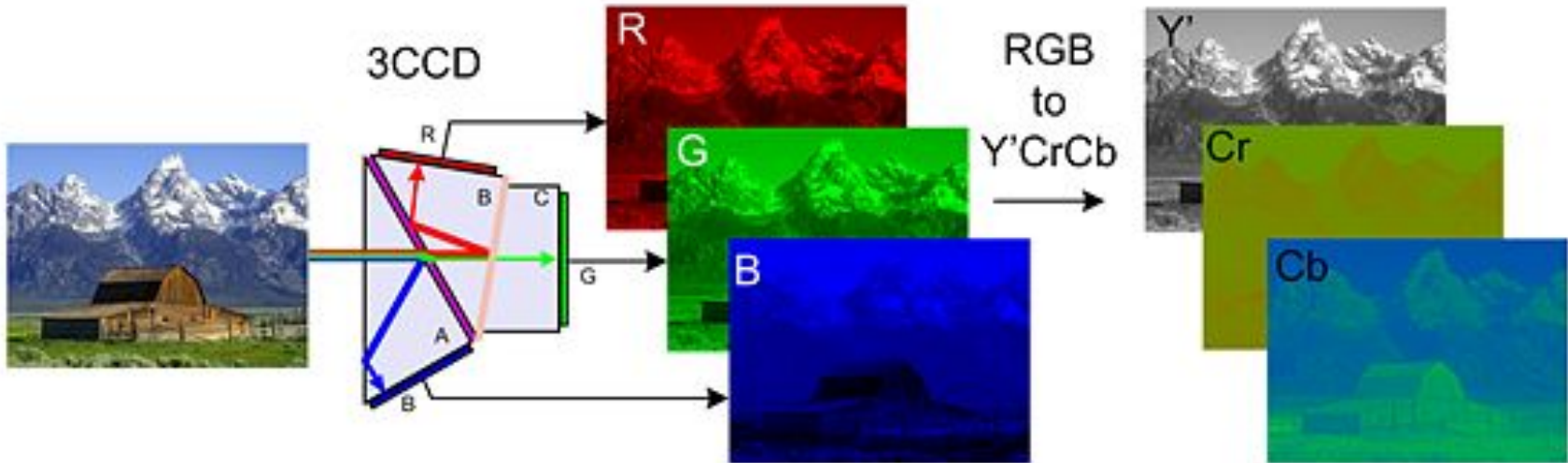


(b) Two dimensions



(c) Three dimensions

3D Arrays



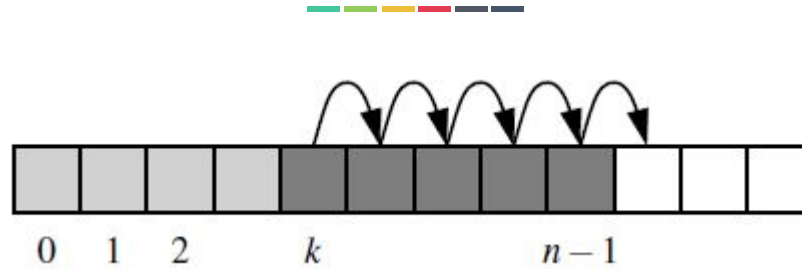


Python-based List Operations



Operation	Running Time
<code>len(data)</code>	$O(1)$
<code>data[i]</code>	$O(1)$
<code>data[i] = val</code>	$O(1)$
<code>c * data</code>	$O(n)$
<code>data.reverse()</code>	$O(n)$
<code>data.sort()</code>	$O(n \log n)$

Adding Elements to a List



- Operation **insert**(k , *value*) requires creating a room to insert a new element at index k of a dynamic array
 - Make room by shifting forward the $(n - 1) - k$ elements $data[k], \dots, data[n-1]$
 - $O(n - k + 1)$ for inserting at index k



Lab 3-1



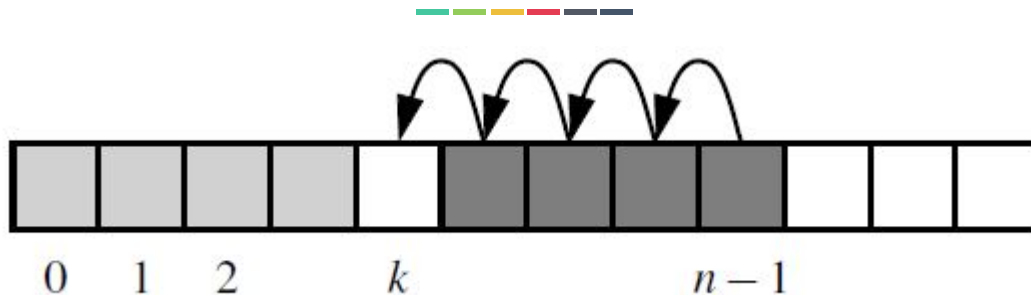
Record the average running time of `insert(k, 20)` in seconds with three different inserting patterns for each of the N calls:

1. Repeatedly insert at the beginning of a list
 2. Repeatedly insert near the middle of a list
 3. Repeatedly insert at the end of the list
- Each pattern starts from an empty list `[]` -> `data = []`

Lab 3-1

	N			
Patterns	10	20	50	100
First Case				
Second Case				
Third Case				

Removing Elements from a List



- Operation **pop**(k) removes the element that is at index $k < n$ of a list
 - Then shifting backward the $(n - 1) - (k + 1)$ elements $data[k+1], \dots, data[n-1]$
 - $O(n - k)$ for removing element at index k

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

10	5	8	7	1			
----	---	---	---	---	--	--	--

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

--	--	--	--	--	--	--	--

Asymptotic Performance



	Add Operation	Running Time	Remove Operation	Running Time
Beginning				
End				
Between				

Data

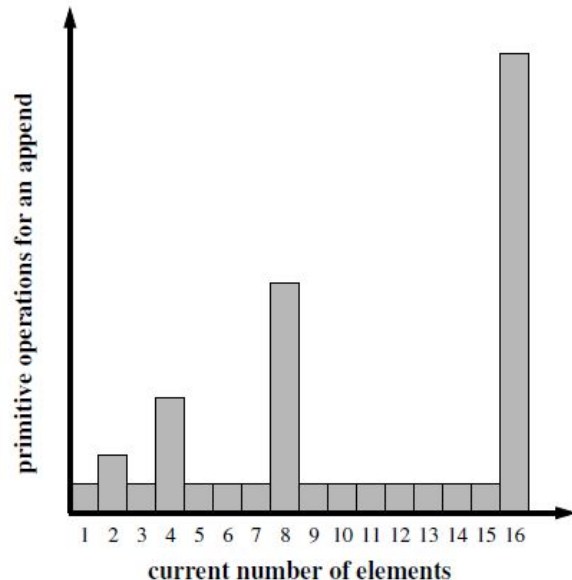
--	--	--	--	--	--	--	--



Amortization (Distribution) Analysis

- Operation **append**(*value*)
 - If an array is not full, copy the value into the array - $O(1)$
 - Otherwise, grow a dynamic array by
copy the old values to a new array - $O(n)$

Does this mean **append**() has $O(n)$?





Lab 3-3



- Operation **append**(*value*)
 - If an array is not full, copy the value into the array - $O(1)$
 - Otherwise, grow a dynamic array by copy the old values to a new array - $O(n)$

Growing array size by 1 or double?

Which option is more efficient in amortized time?